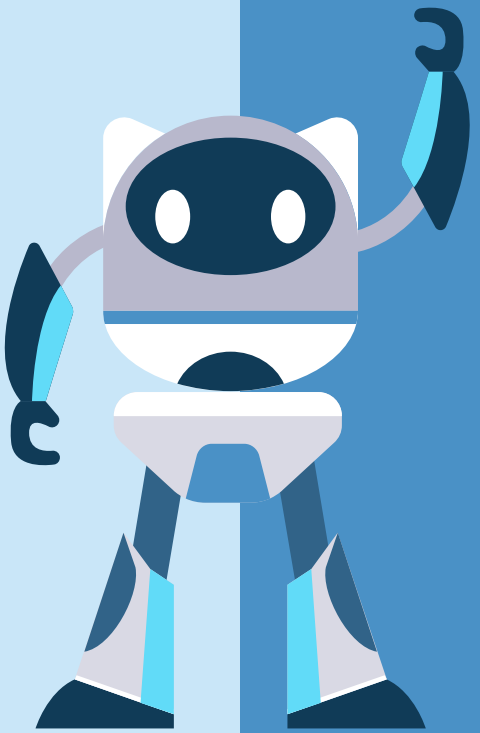
ROS 2 with Python

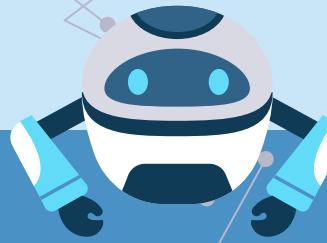
Workspaces, nodes and topic communication

Chapters 4–5 · Edouard Renard



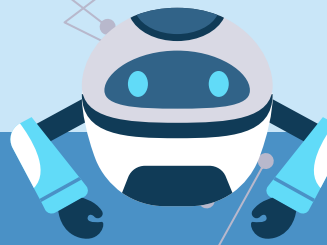
Learning goals

- Create and run a Python node in a ROS 2 workspace.
- Connect Python nodes using typed topic messages.
- Inspect, record and replay topic data.
- Define a custom message and apply topics in Turtlesim.



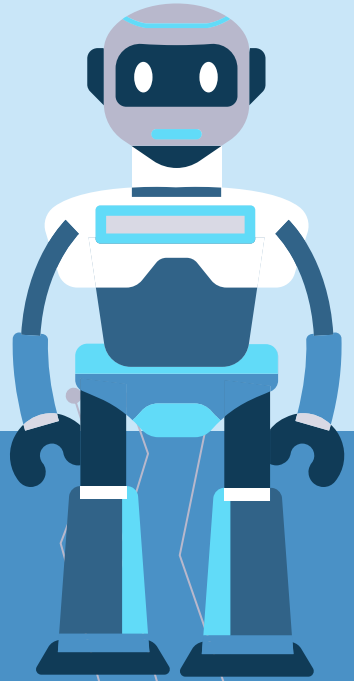
Two-part roadmap

- 01 Workspace, packages and node creation
- 02 Topics, publishers and subscribers
- Examples use our ROS 2 Lyrical environment.
- Prerequisites: ROS 2, `colcon`, Python classes and terminal basics.



01 Workspace, packages and node creation

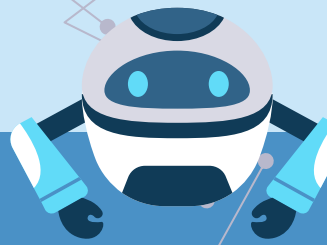
Chapter 4



What is a workspace?

1 · Workspace

- A directory where you organize and build ROS 2 packages.
- `src` contains the packages you write.
- Run `colcon` from the workspace root.
- Example workspace: `~/ros2_ws`



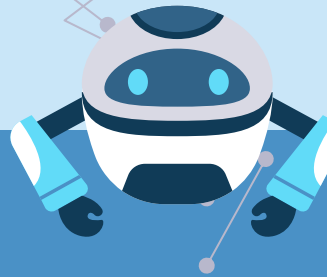
Creating the workspace

1 · Workspace

Terminal BASH

```
$ source /opt/ros/lyrical/setup.bash
$ mkdir -p ~/ros2_ws/src
$ cd ~/ros2_ws
$ colcon build
```

- Source the installed ROS 2 environment first.
- `mkdir -p` creates the workspace and src directory.
- The initial build prepares the workspace even with no packages.



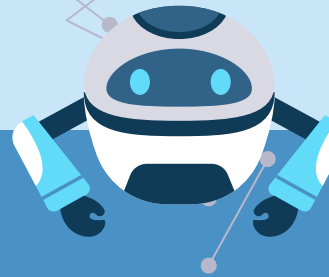
Workspace directories

1 · Workspace

- `src`: your package source files.
- `build`: intermediate build files.
- `install`: installed executables, packages and environment scripts.
- `log`: build logs.
- Edit your source in src; generated directories are build outputs.

~/ros2_ws TREE

```
ros2_ws/
├── src/        # your packages
├── build/      # generated
├── install/    # generated
└── log/        # generated
```



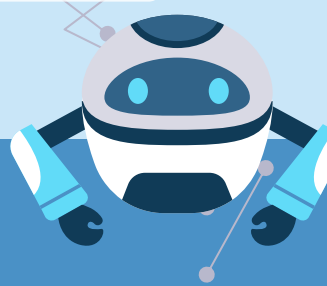
Sourcing the workspace

1 · Workspace

Terminal BASH

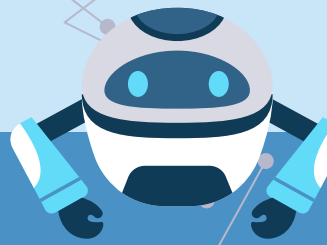
```
$ source /opt/ros/lyrical/setup.bash
$ source ~/ros2_ws/install/setup.bash
```

- Sourcing updates the current terminal's environment.
- Source ROS 2 first, then your workspace.
- Use these commands in each runtime terminal.
- The book adds both lines to `~/.bashrc` for new terminals.

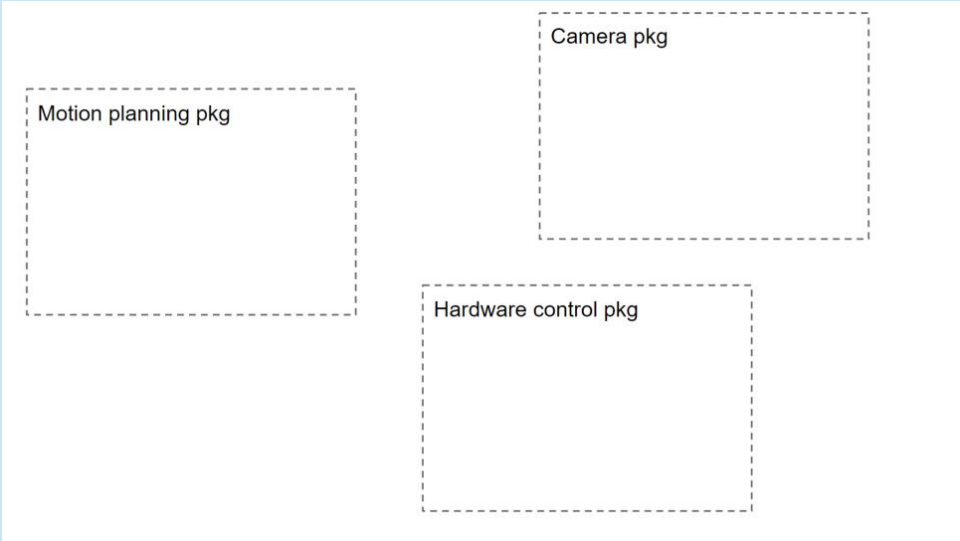


Packages organize an application

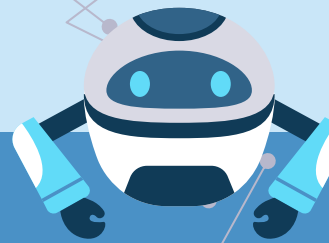
- A package groups a related part of an application.
- Packages declare dependencies and installation information.
- A workspace can contain several packages.
- A package can contain several nodes.



A robot organized into packages

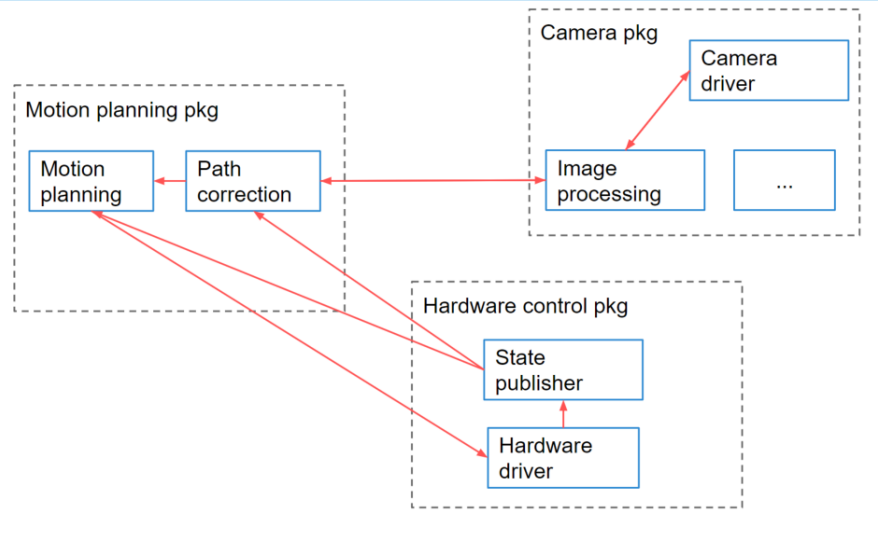


- Camera, motion planning and hardware control each have a clear responsibility.

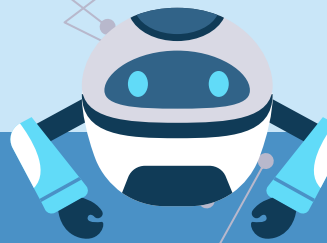


Book pp. 61 · Figure 4.1

Nodes inside the packages



- A node handles a focused responsibility. Nodes communicate to make the whole application work.

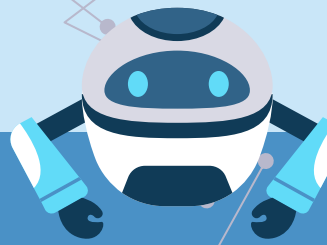


Book pp. 65 · Figure 4.2

Workspace, package, node

2 · Packages and nodes

- Workspace: the development and build directory.
- Package: an installable unit that groups related code and dependencies.
- Node: a participant in the ROS graph with a focused responsibility.
- In these examples, each executable creates one node.



Book pp. 65–69

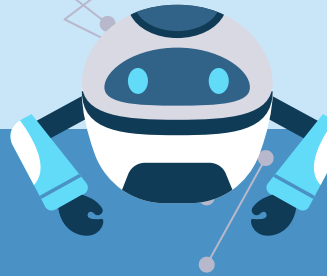
Creating my_py_pkg

3 · Python package

Terminal BASH

```
$ cd ~/ros2_ws/src
$ ros2 pkg create my_py_pkg \
  --build-type ament_python \
  --dependencies rclpy
```

- `my_py_pkg` is the package name.
- `ament_python` selects the Python package build type.
- `rclpy` is the ROS 2 Python client library.



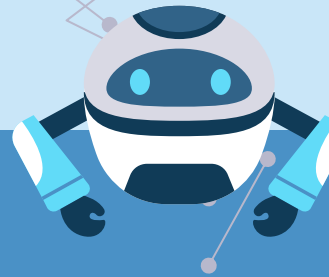
Files you will use

3 · Python package

- Inner `my_py_pkg/`: contains your Python node modules.
- `package.xml`: package metadata and ROS dependencies.
- `setup.py`: installation configuration and executable entry points.
- `setup.cfg`: installation locations for scripts.
- `__init__.py`: identifies the inner directory as a Python package.

~/ros2_ws/src/my_py_pkg — TREE

```
my_py_pkg/
├── my_py_pkg/          # your nodes
│   └── __init__.py
├── package.xml         # dependencies
├── resource/
├── setup.cfg
├── setup.py            # entry points
└── test/
```



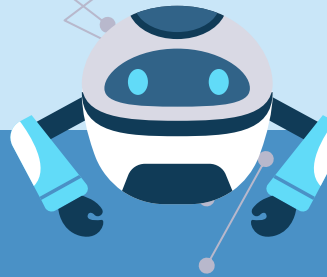
Creating the node file

4 · Python node

Terminal BASH

```
$ cd ~/ros2_ws/src/my_py_pkg/my_py_pkg
$ touch my_first_node.py
$ chmod +x my_first_node.py
```

- Create the file inside the inner `my_py_pkg` directory.
- `touch` creates an empty file.
- `chmod` makes the script directly executable.
- Open this file in your editor for the next code blocks.



Imports and the node class

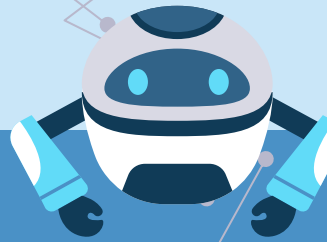
4 · Python node

my_first_node.py — PYTHON

```python
#!/usr/bin/env python3
import rclpy
from rclpy.node import Node

class MyCustomNode(Node):
    def __init__(self):
        super().__init__("my_node_name")
        self.get_logger().info("Hello World")
```

- The shebang selects Python 3 for direct execution.
- `Node` provides ROS capabilities through inheritance.
- `super()` initializes the parent `Node` with its ROS name.
- The logger prints a message when the object is created.



Initialization and spinning

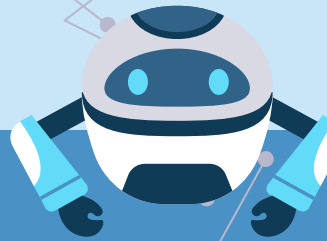
4 · Python node

my_first_node.py — PYTHON

```python
def main(args=None):
    rclpy.init(args=args)
    node = MyCustomNode()
    rclpy.spin(node)
    rclpy.shutdown()

if __name__ == "__main__":
    main()
```

- `init` initializes ROS communication.
- `MyCustomNode()` constructs the node.
- `spin` keeps processing work, including callbacks.
- `shutdown` releases the ROS context.
- The final condition calls main when run directly.



Complete minimal node

4 · Python node

my_first_node.py — PYTHON

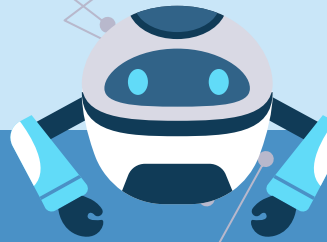
```python
#!/usr/bin/env python3
import rclpy
from rclpy.node import Node

class MyCustomNode(Node):
    def __init__(self):
        super().__init__("my_node_name")
        self.get_logger().info("Hello World")
```

my_first_node.py — PYTHON

```python
def main(args=None):
    rclpy.init(args=args)
    node = MyCustomNode()
    rclpy.spin(node)
    rclpy.shutdown()

if __name__ == "__main__":
    main()
```



Book pp. 67-69

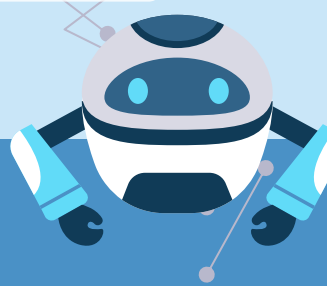
Registering the executable

5 · Build, source, run

setup.py — PYTHON

```python
entry_points={
    "console_scripts": [
        "test_node = my_py_pkg.my_first_node:main",
    ],
},
```

- Edit the existing `entry_points` block in `setup.py`.
- `test_node` is the executable name used by `ros2 run`.
- `my_py_pkg.my_first_node` identifies the Python module.
- The final `:main` selects the function to call.

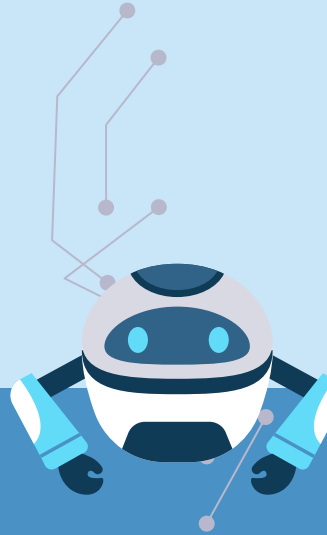


Three names with different roles

5 · Build, source, run

	Example	Where it is defined	Where you use it
File	`my_first_node.py`	Created in `my_py_pkg/my_py_pkg/`	In the `setup.py` entry, without `.py`
Executable	`test_node`	`setup.py console_scripts`	`ros2 run my_py_pkg test_node`
ROS node	`my_node_name`	The constructor, via `super().__init__()`	`ros2 node list`, `rqt_graph`

The three can share one name (e.g. `number_publisher`), but they are still three different things.



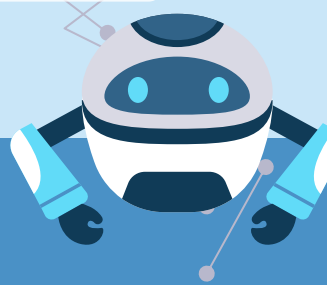
Installing and starting the node

5 · Build, source, run

Terminal BASH

```
$ cd ~/ros2_ws
$ colcon build --packages-select my_py_pkg
$ source install/setup.bash
$ ros2 run my_py_pkg test_node
```

- Build from the workspace root.
- For Python, this installs the package and entry point.
- Source to make the installed package available.
- Expected log: `[my_node_name]: Hello World`
- The node remains alive; Ctrl+C stops it.



A callback every second

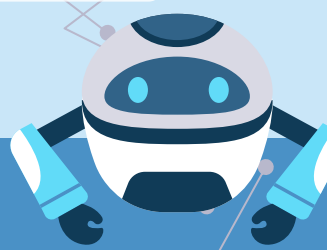
6 · Timers and callbacks

my_first_node.py PYTHON

```python
class MyCustomNode(Node):
    def __init__(self):
        super().__init__("my_node_name")
        self.counter_ = 0
        self.timer_ = self.create_timer(
            1.0, self.print_hello)

    def print_hello(self):
        self.get_logger().info(
            "Hello " + str(self.counter_))
        self.counter_ += 1
```

- Replace the class in `my_first_node.py`.
- 1.0 is the timer period in seconds: about 1 Hz.
- Pass `self.print_hello` without parentheses.
- Each callback prints and increments the counter.
- Keep main unchanged.

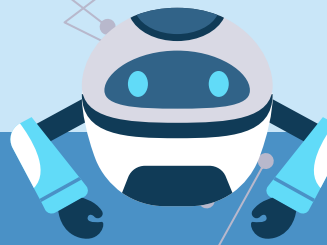


Book pp. 71–73

What happens while a node spins?

6 · Timers and callbacks

- The constructor registers a timer and its callback.
- `spin` lets the executor process callbacks that are ready.
- When the timer is ready, `print_hello` runs.
- After it returns, the executor can process more work.
- Rebuild, source and run: expect Hello 0, Hello 1, Hello 2…



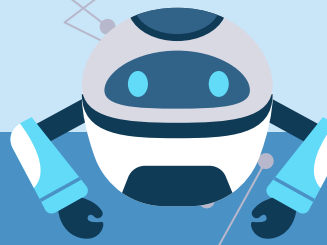
Book pp. 71–73

Checkpoint: predict the behavior

6 · Timers and callbacks

- What changes if the timer period becomes 0.5 seconds?
- Why pass `self.print_hello` instead of `self.print_hello()`?
- What happens if main never calls `spin()`?

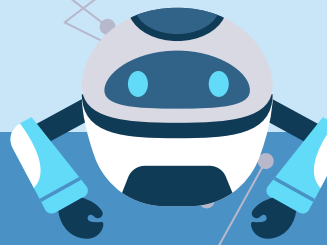
Pause and explain before moving to the answers.



Checkpoint answers

6 · Timers and callbacks

- 0.5 seconds corresponds to a nominal rate of 2 Hz.
- Without parentheses, you pass a callable for later execution.
- With parentheses, you call the function immediately.
- Without spinning or another executor loop, registered callbacks are not processed; this program reaches shutdown and exits.



Inspecting a running node

7 · Node introspection

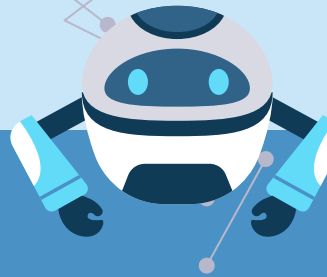
Terminal A — BASH

```
$ ros2 run my_py_pkg test_node
```

Terminal B: source the workspace first — BASH

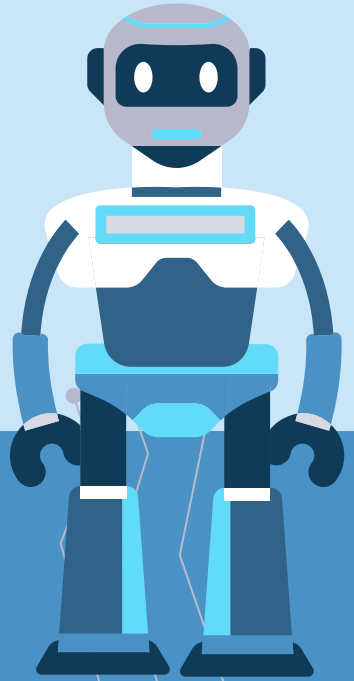
```
$ ros2 node list
$ ros2 node info /my_node_name
```

- `list` shows the names of running nodes.
- `info` shows publishers, subscribers and other endpoints.
- `/rosout` carries log messages.
- You may also see parameter endpoints created by default.



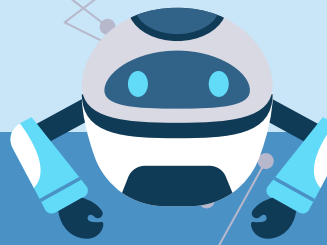
02 Topics, publishers and subscribers

Chapter 5



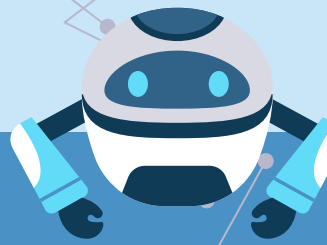
What is a topic?

- A named communication channel for a stream of typed messages.
- A publisher sends messages; a subscriber receives them.
- Both use the same topic name and message type.
- Useful for sensor readings, robot state and repeated commands.



Publishers and subscribers

- A topic can have multiple publishers and subscribers.
- A node can publish and subscribe to several topics.
- Nodes use the topic rather than directly calling one another.
- A publisher can exist without subscribers, and vice versa.
- Compatible communication settings are also needed; these examples use matching defaults.



The message is a data contract

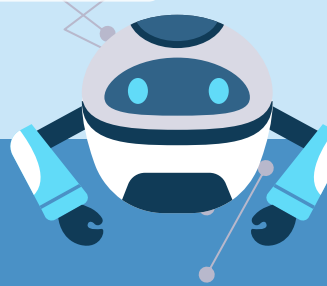
Terminal — BASH

```
$ ros2 interface show example_interfaces/msg/Int64
```

Message definition — MSG

```
int64 data
```

- `example_interfaces` is the interface package.
- `msg` identifies a topic message.
- `Int64` is the type; data is its single field.
- We use this simple example type to learn the mechanics.



Publisher file and dependency

9 · Python publisher

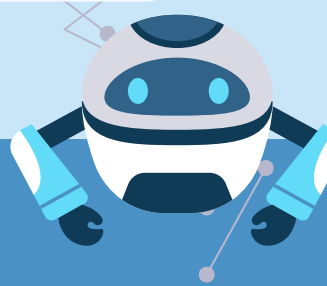
Terminal BASH

```
$ cd ~/ros2_ws/src/my_py_pkg/my_py_pkg
$ touch number_publisher.py
```

Add inside package.xml <package> XML

```
<depend>example_interfaces</depend>
```

- Use the same `my_py_pkg` package.
- Keep its existing `rclpy` dependency.
- `package.xml` declares the interface package dependency.
- The next slides build `NumberPublisherNode`.



Creating the publisher

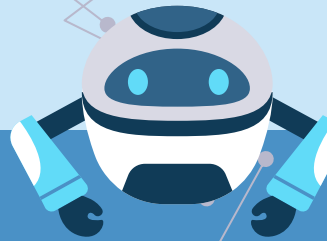
9 · Python publisher

number_publisher.py — PYTHON

```python
from example_interfaces.msg import Int64

class NumberPublisherNode(Node):
    def __init__(self):
        super().__init__("number_publisher")
        self.number_ = 2
        self.number_publisher_ = self.create_publisher(
            Int64, "number", 10)
        self.number_timer_ = self.create_timer(
            1.0, self.publish_number)
        self.get_logger().info(
            "Number publisher has been started.")
```

- Keep the `rclpy` and `Node` imports.
- `Int64` is the message type; number is the topic name.
- 10 sets the QoS history depth, not a rate.
- The timer requests publishing every second.
- The final log shows the node is ready.



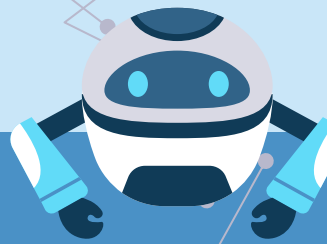
Publishing a message

9 · Python publisher

number_publisher.py PYTHON

```python
def publish_number(self):
    msg = Int64()
    msg.data = self.number_
    self.number_publisher_.publish(msg)
```

- Place this method inside `NumberPublisherNode`.
- Create an instance of the message class.
- Assign 2 to its data field through `self.number_`.
- publish sends that message on the publisher's topic.
- Creating a publisher alone does not send a message.



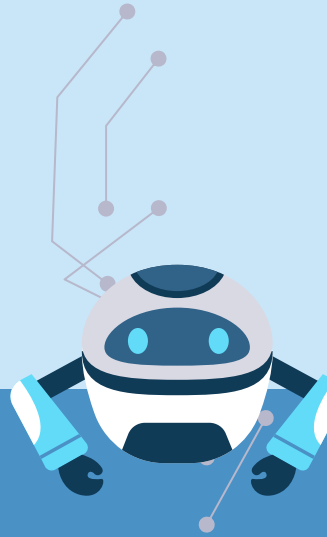
Complete number publisher (1/2)

9 · Python publisher

number_publisher.py — PYTHON

```python
#!/usr/bin/env python3
import rclpy
from rclpy.node import Node
from example_interfaces.msg import Int64

class NumberPublisherNode(Node):
    def __init__(self):
        super().__init__("number_publisher")
        self.number_ = 2
        self.number_publisher_ = self.create_publisher(
            Int64, "number", 10)
        self.number_timer_ = self.create_timer(
            1.0, self.publish_number)
        self.get_logger().info("Number publisher has been started.")
```



Complete number publisher (2/2)

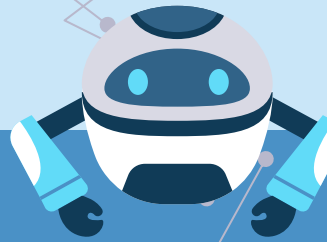
9 · Python publisher

number_publisher.py — PYTHON

```python
    def publish_number(self):
        msg = Int64()
        msg.data = self.number_
        self.number_publisher_.publish(msg)

def main(args=None):
    rclpy.init(args=args)
    node = NumberPublisherNode()
    rclpy.spin(node)
    rclpy.shutdown()

if __name__ == "__main__":
    main()
```



Registering and testing the publisher

9 · Python publisher

setup.py · Add to console_scripts — PYTHON

```
"number_publisher = my_py_pkg.number_publisher:main",
```

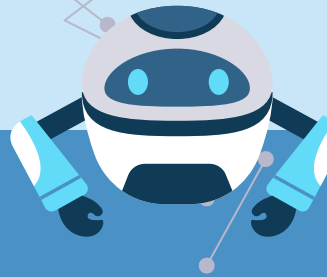
Workspace root — BASH

```
$ colcon build --packages-select my_py_pkg
$ source install/setup.bash
$ ros2 run my_py_pkg number_publisher
```

Terminal 2 (sourced) — BASH

```
$ ros2 topic echo /number
```

- Keep the existing `test_node` entry.
- In a second sourced terminal, echo the topic.
- Expected: repeated data: 2 messages about once per second.



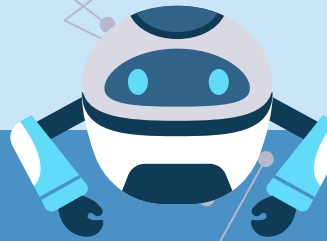
Creating the subscription

10 · Python subscriber

number_counter.py — PYTHON

```python
class NumberCounterNode(Node):
    def __init__(self):
        super().__init__("number_counter")
        self.counter_ = 0
        self.number_subscriber_ = self.create_subscription(
            Int64, "number", self.callback_number, 10)
        self.get_logger().info(
            "Number Counter has been started.")
```

- Create `number_counter.py`; import `rclpy`, `Node` and `Int64`.
- The message type and topic match the publisher.
- The callback receives arriving messages while spinning.
- 10 is the history depth; the final log shows the node is ready.



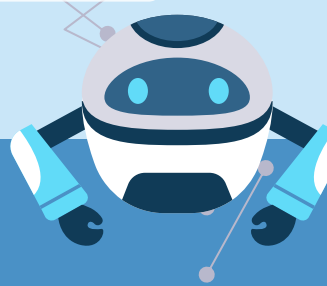
Processing each received number

10 · Python subscriber

number_counter.py — PYTHON

```python
def callback_number(self, msg: Int64):
    self.counter_ += msg.data
    self.get_logger().info(
        "Counter: " + str(self.counter_))
```

- ROS supplies the received message as msg.
- `msg.data` contains the published number.
- The counter stores the sum of received values.
- The type hint helps tools; Python does not enforce it at runtime.



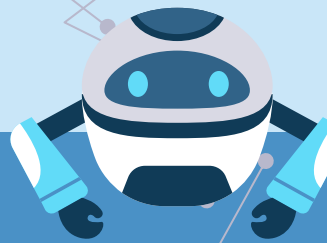
Complete number counter (1/2)

10 · Python subscriber

number_counter.py — PYTHON

```python
#!/usr/bin/env python3
import rclpy
from rclpy.node import Node
from example_interfaces.msg import Int64

class NumberCounterNode(Node):
    def __init__(self):
        super().__init__("number_counter")
        self.counter_ = 0
        self.number_subscriber_ = self.create_subscription(
            Int64, "number", self.callback_number, 10)
        self.get_logger().info("Number Counter has been started.")
```



Complete number counter (2/2)

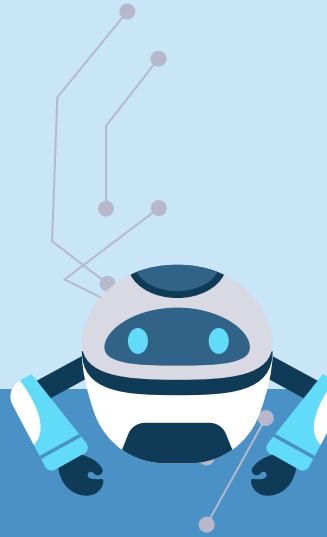
10 · Python subscriber

number_counter.py — PYTHON

```python
    def callback_number(self, msg: Int64):
        self.counter_ += msg.data
        self.get_logger().info(
            "Counter: " + str(self.counter_))

def main(args=None):
    rclpy.init(args=args)
    node = NumberCounterNode()
    rclpy.spin(node)
    rclpy.shutdown()

if __name__ == "__main__":
    main()
```



Running the pair

10 · Python subscriber

setup.py · Add to console_scripts — PYTHON

```
"number_counter = my_py_pkg.number_counter:main",
```

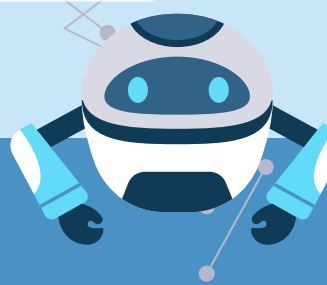
Build and source, then Terminal A — BASH

```
$ ros2 run my_py_pkg number_publisher
```

Terminal B: source first — BASH

```
$ ros2 run my_py_pkg number_counter
```

- Build `my_py_pkg` after adding the executable.
- The publisher sends 2 repeatedly.
- The subscriber logs Counter: 2, then 4, then 6.
- Stop only the publisher: the counter stops changing.
- Restart it: the running subscriber continues its sum.



Tracing one message

10 · Python subscriber

number_publisher node

topic

number_counter node

1

Timer fires

The 1.0 s timer makes `publish_number` ready.

2

Publish

`publish_number` creates `Int64(data=2)` and publishes it.

3

Topic

The message travels on `/number`.

4

Receive

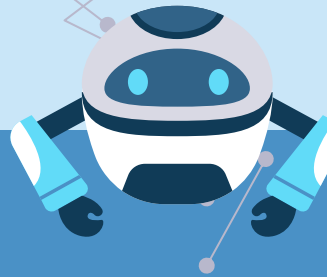
The executor runs `callback_number` with the message.

5

Update

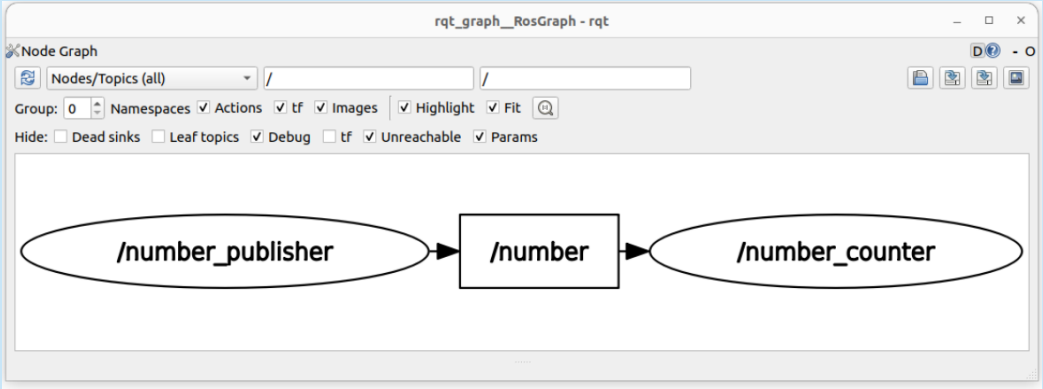
It adds `msg.data` to `counter_` and logs the new total.

This cycle repeats on every timer tick, about once per second.

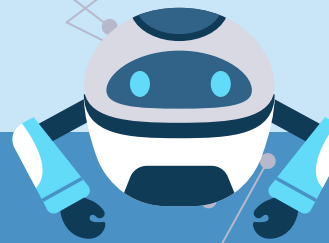


Seeing the communication graph

11 · Topic introspection



- Run both nodes, then run `rqt_graph`.
- Choose Nodes/Topics (all). Refresh if needed.
- Arrows show the publisher and subscriber.



Book pp. 102 · Figure 5.7

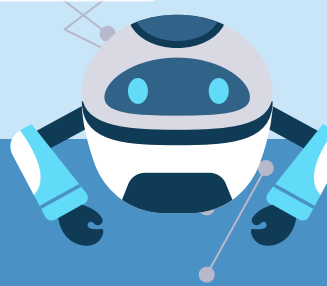
Names, types and endpoints

11 · Topic introspection

Terminal BASH

```
$ ros2 topic list
$ ros2 topic info /number
$ ros2 interface show example_interfaces/msg/Int64
```

- `list`: which topics exist?
- `info`: what type and how many endpoints?
- Expected for our pair: 1 publisher and 1 subscription.
- `interface show`: which fields can the message contain?



Testing from the terminal

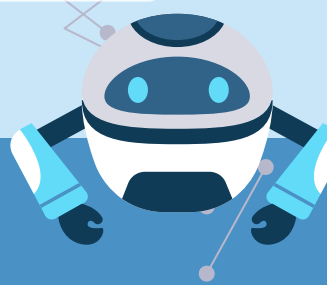
11 · Topic introspection

Terminal BASH

```
$ ros2 topic echo /number

# Stop the original publisher before this test
$ ros2 topic pub -r 2.0 /number \
  example_interfaces/msg/Int64 "{data: 7}"
```

- `echo` acts as a subscriber and displays messages.
- `pub` acts as a publisher for a quick test.
- `-r 2.0` requests two messages per second.
- Keep `number_counter` running: each received message adds 7.
- The message fields use YAML syntax.

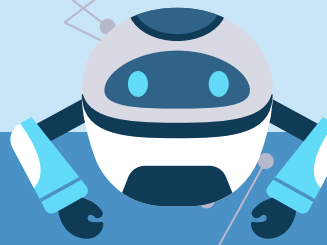


Checkpoint: communication

11 · Topic introspection

- Can one node have a publisher and a subscriber?
- Does `number_counter` count messages or sum their values?
- Which command tests a publisher without writing a subscriber?
- Which command tests a subscriber without writing a publisher?

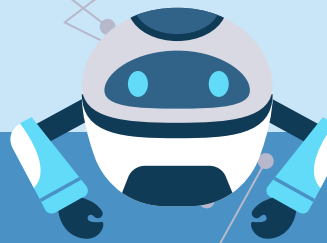
Pause and explain before moving to the answers.



Checkpoint answers: communication

11 · Topic introspection

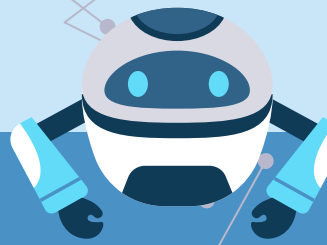
- Yes. A node can hold several publishers and subscribers; the Turtlesim controller has one of each.
- It sums values: each callback adds `msg.data` to `counter_`.
- `ros2 topic echo /number` acts as a test subscriber for a publisher.
- `ros2 topic pub /number` … acts as a test publisher for a subscriber.



What is a bag?

12 · Bags

- A recording of topic messages and their timing information.
- Useful for repeating an experiment or developing without live hardware.
- Recording stores messages; playback publishes recorded messages.
- Subscribers can process playback as a source of topic data.

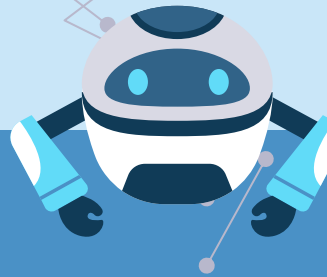


Recording /number

Terminal BASH

```
$ mkdir -p ~/bags
$ cd ~/bags
$ ros2 bag record /number -o bag1
```

- Start `number_publisher` in another terminal first.
- The recorder subscribes to `/number` and stores arriving messages.
- Let it run for several seconds.
- Press Ctrl+C to finish writing the recording.
- bag1 is the output directory.

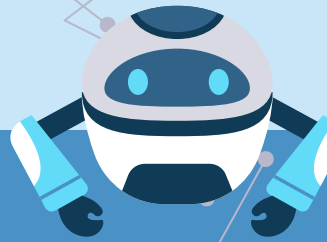


Replaying the recording

Terminal BASH

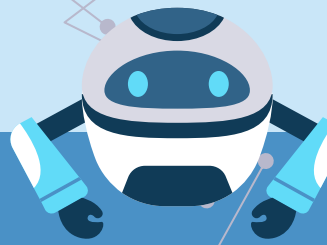
```
$ ros2 bag play ~/bags/bag1
```

- Stop `number_publisher` before replaying.
- Keep `number_counter` running in another sourced terminal.
- Playback publishes the recorded numbers.
- The counter increases as messages arrive, then stops at the end.
- Use a fresh subscriber process if you want its sum to start at zero.



Choosing a message type

- First describe the data you need to communicate.
- Look for an existing message with the right meaning and fields.
- `geometry_msgs/Twist` represents linear and angular velocity.
- `sensor_msgs/Image` represents image data.
- Create a custom interface when existing messages do not fit.



Inspecting and using an interface

13 · Existing interfaces

Terminal — BASH

```
$ ros2 interface show geometry_msgs/msg/Twist
```

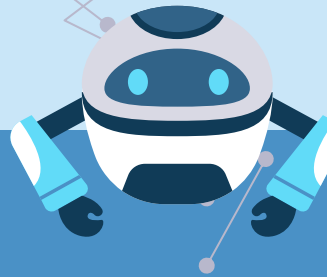
In package.xml — XML

```
<depend>geometry_msgs</depend>
```

In Python — PYTHON

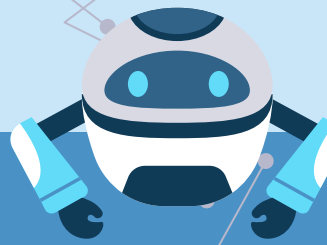
```
from geometry_msgs.msg import Twist
```

- Inspect the fields before writing code.
- Declare the dependency in the package containing your node.
- Import the message and use it in a publisher or subscription.
- Primitive example messages are useful for learning; choose semantically meaningful types for applications.



An interface package

- Keep custom message definitions in a dedicated package.
- Use `ament_cmake` to generate the interfaces.
- The Python nodes stay in the `ament_python` package.
- Our interface package: `my_robot_interfaces`.
- Our message: `HardwareStatus.msg`.

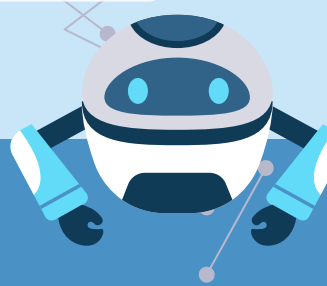


Creating the interface package

Terminal BASH

```
$ cd ~/ros2_ws/src
$ ros2 pkg create my_robot_interfaces \
  --build-type ament_cmake
$ cd my_robot_interfaces
$ mkdir -p msg
```

- The build type is explicit here.
- Create message definitions inside msg.
- This package contains interfaces rather than nodes.
- You can leave the generated empty `src/include` directories unused.

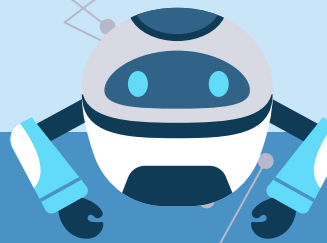


Defining HardwareStatus.msg

msg/HardwareStatus.msg MSG

```
int64 version
float64 temperature
bool are_motors_ready
string debug_message
```

- Save as `msg/HardwareStatus.msg` in the interface package.
- Each line gives a field type and name.
- Use UpperCamelCase for the message filename.
- These fields describe hardware status meaningfully.
- Document units such as degrees Celsius for temperature.



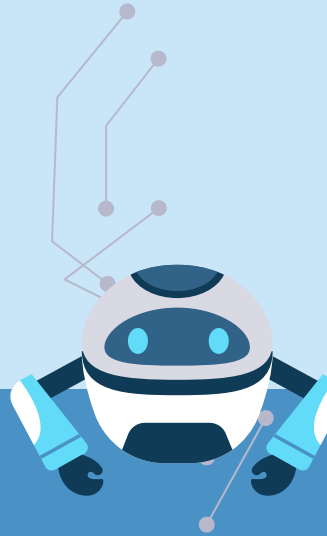
Interface package dependencies

14 · Custom interfaces

my_robot_interfaces/package.xml XML

```xml
<build_depend>rosidl_default_generators</build_depend>
<exec_depend>rosidl_default_runtime</exec_depend>
<member_of_group>rosidl_interface_packages</member_of_group>
```

- Add these inside the interface package's `package.xml`.
- Keep `buildtool_depend` for `ament_cmake`.
- Generators produce language support from the definitions.
- Runtime supplies dependencies for generated interfaces.
- The group declaration identifies an interface package.



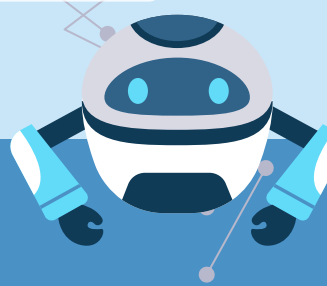
Generating the message

my_robot_interfaces/CMakeLists.txt — CMAKE

```
find_package(ament_cmake REQUIRED)
find_package(rosidl_default_generators REQUIRED)

rosidl_generate_interfaces(${PROJECT_NAME}
  "msg/HardwareStatus.msg"
)
ament_export_dependencies(rosidl_default_runtime)
ament_package()
```

- Edit `CMakeLists.txt`, retaining `cmake_minimum_required` and project above this block.
- List each `.msg` file inside `rosidl_generate_interfaces`.
- No commas are needed between message paths.
- Keep `ament_package()` at the end.

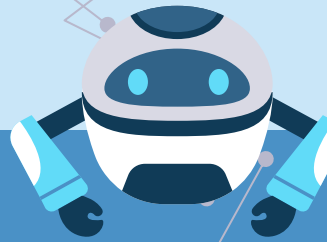


Building and inspecting the interface

Terminal BASH

```
$ cd ~/ros2_ws
$ colcon build --packages-select my_robot_interfaces
$ source install/setup.bash
$ ros2 interface show \
  my_robot_interfaces/msg/HardwareStatus
```

- Build generates a Python message class from the definition.
- Source before importing or inspecting it.
- Expected output lists all four fields.
- The definition is shared by publishers and subscribers.



Using HardwareStatus in Python

14 · Custom interfaces

Add to my_py_pkg/package.xml — XML

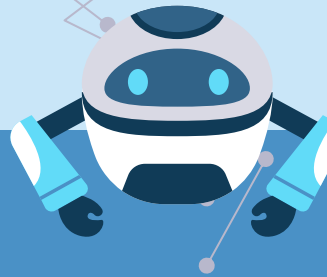
```xml
<depend>my_robot_interfaces</depend>
```

your node (.py) — PYTHON

```python
from my_robot_interfaces.msg import HardwareStatus

# In a node constructor
self.status_pub_ = self.create_publisher(
    HardwareStatus, "hardware_status", 10)
```

- Declare the interface package as a node-package dependency.
- Import the generated class.
- Use `HardwareStatus` as the publisher's type.
- A subscriber must use the same message type and topic.

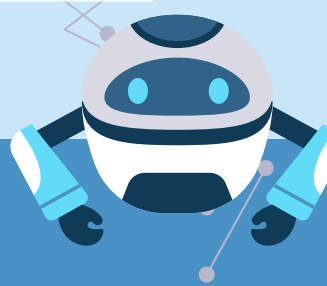


Filling and publishing the fields

your node (.py) PYTHON

```python
msg = HardwareStatus()
msg.version = 1
msg.temperature = 34.5
msg.are_motors_ready = True
msg.debug_message = "Ready"
self.status_pub_.publish(msg)
```

- Run this inside a callback, such as a timer callback.
- Assign each field using its declared type.
- Rebuild `my_py_pkg` and source after editing.
- Inspect with `ros2 topic echo /hardware_status`.



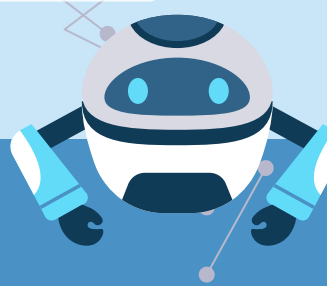
Receiving the custom message

In a node constructor — PYTHON

```python
self.status_sub_ = self.create_subscription(
    HardwareStatus, "hardware_status",
    self.callback_status, 10)

def callback_status(self, msg: HardwareStatus):
    self.get_logger().info(
        f"Temperature: {msg.temperature} C")
```

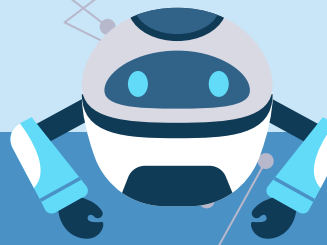
- Import `HardwareStatus` in this node too.
- The callback reads meaningful named fields.
- Place `callback_status` inside the node class.
- This uses the same subscription pattern as `number_counter`.



Challenge: feedback controls velocity

15 · Turtlesim challenge

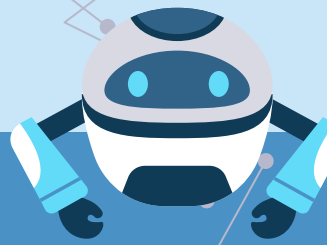
- Create a Python node that makes the turtle move in a circle.
- Subscribe to `/turtle1/pose` to read its x coordinate.
- Publish `Twist` messages on `/turtle1/cmd_vel`.
- When x < 5.5: `linear.x` = 1.0 and `angular.z` = 1.0.
- Otherwise: set both values to 2.0.



Plan before coding

15 · Turtlesim challenge

- Inspect the two topic types and their fields.
- Create a `turtle_controller` package and Python node.
- Add a publisher and subscriber in the constructor.
- In the pose callback, choose the speed and publish the command.
- Try your implementation before reviewing the following solution.



Solution: package and imports

15 · Turtlesim challenge

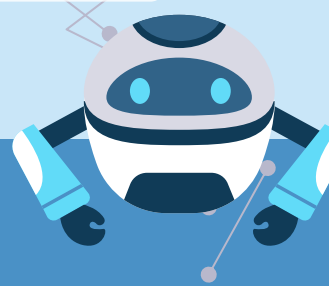
Terminal BASH

```
$ cd ~/ros2_ws/src
$ ros2 pkg create turtle_controller \
  --build-type ament_python \
  --dependencies rclpy geometry_msgs turtlesim
```

In turtle_controller/controller.py PYTHON

```
import rclpy
from rclpy.node import Node
from geometry_msgs.msg import Twist
from turtlesim.msg import Pose
```

- Create `controller.py` in the inner `turtle_controller` directory.
- `Twist` carries velocity commands.
- `Pose` carries the turtle's current pose.
- The dependency list covers all three imports.

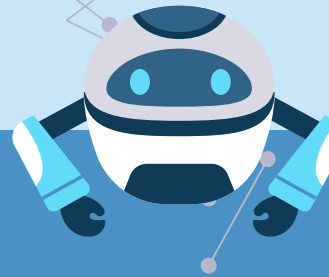


Solution: publisher and subscriber

controller.py PYTHON

```python
class TurtleControllerNode(Node):
    def __init__(self):
        super().__init__("turtle_controller")
        self.cmd_vel_pub_ = self.create_publisher(
            Twist, "/turtle1/cmd_vel", 10)
        self.pose_sub_ = self.create_subscription(
            Pose, "/turtle1/pose",
            self.callback_pose, 10)
```

- One node contains both communication endpoints.
- The subscriber supplies feedback about position.
- The publisher sends commands to Turtlesim.
- No timer is required for this exercise.

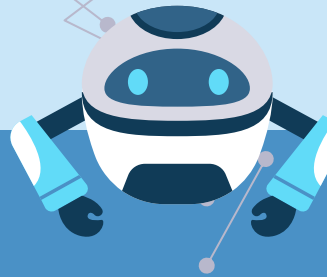


Solution: the feedback callback

controller.py PYTHON

```python
def callback_pose(self, pose: Pose):
    speed = 1.0 if pose.x < 5.5 else 2.0
    cmd = Twist()
    cmd.linear.x = speed
    cmd.angular.z = speed
    self.cmd_vel_pub_.publish(cmd)
```

- The current position selects the velocity.
- `linear.x` controls forward speed.
- `angular.z` controls turning speed.
- Other `Twist` components remain at their default zero values.
- Each pose callback produces a new command.



Solution: main and registration

controller.py — PYTHON

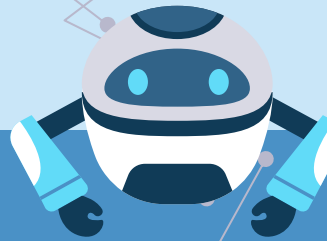
```python
def main(args=None):
    rclpy.init(args=args)
    node = TurtleControllerNode()
    rclpy.spin(node)
    rclpy.shutdown()

if __name__ == "__main__":
    main()
```

setup.py · console_scripts entry — PYTHON

```
"controller = turtle_controller.controller:main",
```

- Place main after the class in `controller.py`.
- Add the final string to `setup.py`, not to `controller.py`.
- The entry maps the executable controller to main.



Running the controller

15 · Turtlesim challenge

Terminal — BASH

```
$ cd ~/ros2_ws
$ colcon build --packages-select turtle_controller
$ source install/setup.bash
```

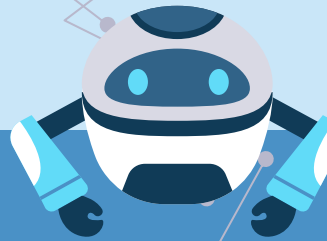
Terminal A — BASH

```
$ ros2 run turtlesim turtlesim_node
```

Terminal B: source first — BASH

```
$ ros2 run turtle_controller controller
```

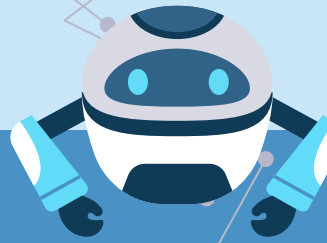
- Expected: circular motion with speed changing across x = 5.5.
- The command depends on measured position: a feedback loop.
- Use `rqt_graph` to inspect both topics.
- Both velocity values scale together, preserving their ratio.



Final checkpoint

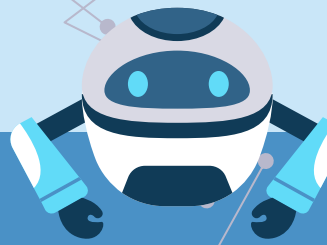
- Why can one node subscribe to `Pose` and publish `Twist`?
- Why can a bag replace the live number publisher?
- Why use `ament_cmake` for an interface package?

Pause and explain before moving to the answers.



Final checkpoint answers

- Each endpoint has its own type: one node can subscribe to `Pose` and publish `Twist`.
- Playback republishes the recorded messages on `/number`, so `number_counter` receives them like live data.
- The interface-generation tools (`rosidl_default_generators`) run through CMake, so the package uses `ament_cmake`.



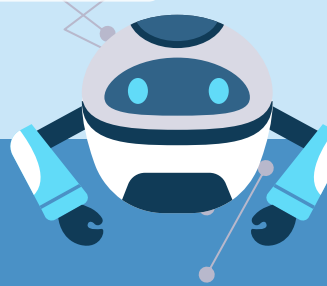
The build–source–run workflow

Study reference

Terminal BASH

```
$ cd ~/ros2_ws
$ colcon build --packages-select my_py_pkg
$ source install/setup.bash
$ ros2 run my_py_pkg number_publisher
```

- New node: create the module and register its entry point.
- New dependency: update `package.xml`.
- Changed code or setup: rebuild the package.
- New terminal: source ROS 2 and the workspace.



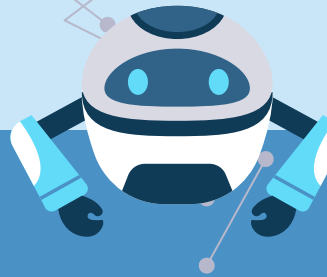
Node and topic commands

Study reference

Terminal BASH

```
$ ros2 node list
$ ros2 node info /number_publisher
$ ros2 topic list
$ ros2 topic info /number
$ ros2 topic echo /number
$ ros2 interface show example_interfaces/msg/Int64
$ rqt_graph
```

- Use node commands to inspect participants.
- Use topic commands to inspect communication.
- Use interface show to understand the data contract.
- Use `rqt_graph` to see connections.



Testing and recording commands

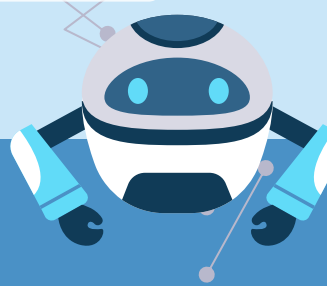
Study reference

Terminal BASH

```
$ ros2 topic pub -r 2.0 /number \
  example_interfaces/msg/Int64 "{data: 7}"

$ ros2 bag record /number -o bag1
$ ros2 bag play ~/bags/bag1
```

- `pub` generates test messages from the terminal.
- `record` captures incoming topic messages.
- `play` republishes a saved recording.
- Stop the original publisher for a playback-only test.



Sources and design credits

- Teaching source: Edouard Renard, ROS 2 from Scratch, Packt, 2024.
- Chapter 4: Writing and Building a ROS 2 Node, pp. 57–82.
- Chapter 5: Topics, pp. 83–117.
- Figures 4.1, 4.2 and 5.7 reproduced from the supplied book.
- Presentation template by Slidesgo. Original template assets retained.

